

Namespace

All objects, which you inserted into a *Frame*, plus the names of the built-in attributes and methods and the names of the *user-defined attributes* of the *Frame*, form a **namespace**.

Remarks

All objects, which you inserted into a *folder*, plus the names of the built-in attributes and methods of the *folder*, also form a **namespace**.

The same applies for *objects* which provide built-in attributes and methods and for which you created *user-defined attributes* of the *object*.

Within this **namespace**, you cannot use the same name more than once, i.e., each name has to be unique! You can use identical names in different name-spaces though, as then *Plant Simulation* differentiates them using their paths.

See also

[Name \[general description\]](#)

[isNameUnique \[SimTalk\] - identifier](#)

Constant Values

Constant Values

Constants contain values that do not change.

Remarks

You can assign constant values of the following data types:

Boolean [SimTalk] - value	Real [SimTalk] - value
Integer [SimTalk] - value	String [SimTalk] - value
pi [SimTalk] - value	

See also

[Simple Values](#)

Boolean [SimTalk] - value

You can assign `true` or `false` to a constant value of data type *boolean*.

Example

```
order_done := true
```

See also

[Boolean \[SimTalk\] - data type](#)

Integer [SimTalk] - value

You can assign numbers from 0 to 9 with an optional leading sign, namely plus (+) or minus (-), to a constant value of data type *integer*. This is also called *integer constant*.

Remarks

The range for constant *integer* values is –9223372036854775807 to 9223372036854775807.

Example

```
order_number := -112304
```

See also

[Integer \[SimTalk\] - data type](#), data type

pi [SimTalk] - value

The constant value π is the ratio of the circumference of a circle to its diameter. It has the data type *real*.

Example

```
print pi // 3.141592653589793
```

Real [SimTalk] - value

You can assign one or several digits after the decimal point to a constant value of data type *real*. This is also called a *real constant*.

Remarks

In exponential notation the constant value consists of a mantissa followed by the letter E and an exponent. You can, for example, also specify the number 100 as 1.0e2 (1.0 times the second power of 10).

Note

The value range of the exponent is limited to the interval [307, -307].

Note

You recognize a constant value of data type *real* by the **decimal point**. If it is missing, the constant value has the data type *integer* with a limited value range.

The data types *time*, *date*, and *dateTime* do not support constant values. Instead, use *real* or *integer* constant values, or the data type conversion functions [str_to_time](#), [str_to_date](#), and [str_to_dateTime](#).

Example

```
Conveyor.Speed := 2.5  
Avogadro := 6.5e23  
epsilon := 1.0e-6 // 0.000001
```

See also

Real [SimTalk] - data type

String [SimTalk] - value

You can **assign a string** consisting of letters, numbers and special characters within quotation marks "" to a constant value of data type *string*.

Remarks

When working with constants of data type *string*, i.e., a sequence of characters, keep this in mind:

- You can **assign a string** consisting of letters, numbers and special characters within quotation marks (") to a constant value of data type *string*. This is also called a *string constant*.

```
priority: = "Urgent"
```

- To **use quotation marks (quotes)** within the string, you have to protect the quotation marks by typing in a backslash \ in front of them. Otherwise *Plant Simulation* considers the quotation mark to be the end of the constant value.

```
print "It is \"very\" urgent."
-- outputs It is "very" urgent.
```

- To **specify a line break** within the string, terminate the line of text at this position with a backslash \ and continue the *string constant* in the next line.

```
var address : string
address := "Frank Jones\
Halford Lane 9\
Cedar Rapids, IA 52409"

print address
-- outputs
Frank Jones
Halford Lane 9
Cedar Rapids, IA 52409
```

- Protect a backslash** within a string by typing a second backslash \\. This is necessary if the *string literal* ends with a backslash.

```
a := "It is \"very\" urgent." -- the backslash protects the quotation
marks
Path := "C:\Temp"           -- no need to protect the backslash here
Path := "C:\\\" + FolderName -- protected backslash at the end of the
string
```

- A constant value of data type *string* can contain 1024 characters at most. You can put together *strings* of arbitrary length by **linking together individual strings** with the **+** operator though.

```
var s : string
s := "This is a long string," +
" formed by linking together several" +
" sub-strings."
print s
-- outputs This is a long string, formed by linking together several sub-strings.
```

See also

[String \[SimTalk\] - data type](#)

Protect quotation marks in [Why Use SimTalk 2.0?](#)

Variables

Variables [SimTalk]

Local variables, *global variables*, and *parameters* store values over time and allow to access them at a later point in time.

You can change the value of a variable at any time according to your modeling needs.

See also

[Local Variables](#)

[Global Variables](#)

[Parameters \[SimTalk\]](#)

Local Variables

If you want to reuse the result of a calculation again at a later point in time, you can save that result to a *local variable*. Use a *local variable* if you do not need the value again after the method call.

Remarks

The name of a *local variable* is only known in the *Method* in which you declare it. Other *Methods* cannot access this *local variable*. For each *Method* call *Plant Simulation* creates a new set of *local variables*. If a *Method* calls itself, for example, *Plant Simulation* creates a new set of *local variables* and the *Method* can only access this new set. Only when the call is finished, the previous *local variables* are available again with the previous values.

You have to **declare a local variable within the source code** before you can use it.

Before you assign a value to the *local variable*, it has an *initial value*.

- It is **0** for all *numerical* data types.
- It is **false** for the data type *boolean*.
- It is an **empty string** "" for the data type *string*.
- It is **void** for the data types *object* and *list/table*.
- It is **1900/01/01** for the data type **date** and **1900/01/01 00:00:00.0000** for the data type **dateTime**.

For models which you initially created in *Plant Simulation* version 13 or older, the initial value is a different one and depends on the version.

If you need to access the saved value later on, use the object **Variable**  instead, compare **global variables**.

Data Types in Local Variables

Use any of the data types provided in *SimTalk* in your local variables.

Remarks

You can use these data types in a local variable: **Integer**, **Real**, **Length**, **Speed**, **Acceleration**, **Weight**, **Time**, **Date**, **DateTime**, **Json**, **Boolean**, **String**, **Object**, **Table**, **List**, **Stack**, **Queue**, and **Any**.

For the data types *stack*, *queue*, *list* and *table* you have to type the data type or data types of the columns in between brackets, [data type]. These local variables slightly differ from the objects, which you insert into a *Frame*. As opposed to the objects, they only provide built-in methods for *Instantiation*, *State*, *Access*, and *Order*.

Data type	Shares built-in properties of the object
List [SimTalk] - data type	DataList 
Queue [SimTalk] - data type	Properties of the DataQueue 
Stack [SimTalk] - data type	Properties of the DataStack 
Table [SimTalk] - data type	DataTable [object] 

Note

Before first accessing these data types, you have to create the *local variable* with the method **create** or you have to assign a value to the variable.

Example

```
var l: list[string]
l.create
l.insert(1, "Hello")
```

Plant Simulation automatically initializes all local variables. The initial value depends on the data type:

Data Type	Initial Value
<i>integer</i> , <i>real</i> , <i>length</i> , ... [SimTalk]	0
<i>boolean</i> [SimTalk]	false
<i>string</i> [SimTalk]	""
<i>object</i> [SimTalk]	void
<i>time</i> [SimTalk]	0:00:00

Compare

[Data Types \[SimTalk\]](#)

Declare Local Variables in the Source Code

You can declare a *local variable* anywhere within the source code.

Remarks

Start to declare your variable with the keyword **var**, followed by one or several **identifiers** for the local variables you are defining. After that you can type a **colon** and define the **data type** of the local variable.

Example

```
var x, y: integer
```

You can also assign a value to the *local variable* when you declare it.

A *local variable*, which you declare like this, is visible from the line, in which you defined it, to the end of the source code.

A *local variable* is initialized with an initial value when calling the *Method*. Compare [Data Types in Local Variables](#).

Example

```
var a: length := Track1.Length
var b: Length := Track2.Length
var c: Length := sqrt(a*a + b*b)
print c
```

If you assign a value to the *local variable* while you declare it, declaring the data type is optional. The data type of the variable is then determined by the assigned value.

In the example the *local variables* named *a* and *b* will take the data type *length*. The variable *c* will take the data type *real*, as the function *sqrt* returns a value of data type *real*.

Example

```
var a := Track1.length
var b := Track2.length
var c := sqrt(a*a + b*b)
print c
```

Plant Simulation sets *local variables* to the initial value when calling the *Methods*. If you declare a *local variable* within a **loop**, the initial value will not be set anew with each loop pass, but the *local variable* keeps its previous value. If you assign a value to the *local variable* within the declaration, *Plant Simulation* executes the assignment anew with each loop pass.

Example

```
// the loop outputs: 0 0 --> 1 0 --> 2 0
for var i := 1 to 3
    var x: integer
    var y: integer := 0
    print x, " ", y
    x += 1
    y += 1
next
```

A *local variable*, which is declared within an **if-statement**, is also visible after the **if-statement**. This also applies if the instructions within the **if-statement** are not executed. In this case the *local variable* has the initial value, even if the variable declaration contains an assignment.

Example

```
if 1 = 2
    var x: integer := 1
end
print x // outputs 0
```

This also applies to **loops**: If the loop was never executed because the loop condition was not fulfilled the first time already, you can still access the *local variable*, which you declared within the loop, after the loop. In that case the *local variable* contains the initial value.

Example

```
for var y := 1 to DataTable.yDim
    var i : integer
    i := DataTable[1,y]
next
print i // outputs 0 if the DataTable is empty
```

Video on YouTube

<https://youtu.be/MXDcx2yplxc?si=clCVJfstejBUiwXM&t=62>

Global Variables

A *global variable*, which contains a non-fixed value, can be accessed by any *Method* in the simulation model. Data stored in a *global variable* by a *Method* will still exist after the execution of that *Method* has finished.

Remarks

Use *global variables* (or entries in lists) to keep data for extended periods of time during a simulation run.

Plant Simulation provides the object **Variable**  as a *global variable*. Any *Method* can access the *global variable* by its name and an absolute or relative path pointing to it.

If you do not need to access the saved value later on, use a **local variable** instead.

Example

```
MyVariable := Transporter:1.CurrentSpeed -- assign a new value to the
global variable 'MyVariable'
print MyVariable                          -- read out the value and print
it into the Console window

print &MyVariable.DataType               -- print the data type of 'MyVariable'
var obj : object := &MyVariable          -- store a reference to 'MyVariable' in a
local variable of data type object
```

Parameters in SimTalk

Parameters [SimTalk]

You can pass values to a *Method* when this *Method* is called during the simulation run. These values are called **arguments**.

Remarks

You have to declare the same number of **parameters** within the *Method* and these parameters have to have the same data types as the passed arguments. Exceptions are *integer* and *real* values. *Plant Simulation* automatically converts these.

In the example below **x** is the name of the **parameter** and **123** is the **value** of the **argument**.

```
param x: integer := 123
```

Note

When converting *real* values to *integer* values, *Plant Simulation* deletes numbers after the decimal point. This may lead to your *Method* to not behave as you would expect. In general, it is a good idea to avoid automatic type conversion.

See also

[Declare Parameters](#)

[Declare Default Arguments](#)

[param \[SimTalk\]](#)

[Declare the Function Result of a Method](#)

[Programming a Method](#)

[Video on YouTube](#)

<https://youtu.be/5t-wLNmKpbU?si=Q-8tGoRXDly3nd2i&t=34>

Declare Parameters

If the *Method* needs **parameters**, you have to declare these parameters at the start of the source code.

Remarks

Start the declaration with the keyword **param**, then enter the **identifier**, followed by a **colon**, and the **data type**. The identifier serves for accessing the parameter later on.

If several parameters in sequence have the same data type, you can list all identifiers of these parameters, separated by commas, followed by a colon and the data type.

If you want to declare several parameters with different data types, separate the declarations (name, colon, and data type) with commas.

```
param repeats: integer           // single parameter of
data type integer
param minimum, maximum: real    // two parameters of data
type real
param rpm: real, tool: string    // two parameters with
different data types
param workpiece: string, Islength, shorten: real // three parameters with
different data types
```

In this *Method* the parameters can be used like **local variables**. The initial value is set by the caller. This way the *Method* can react to different values.

Type the keyword **byref** in front of the parameter to not pass the value, but a reference to this variable. Changes of the parameter also affect the caller of the *Method*.

For the data types *stack*, *queue*, *list*, and *table* you just have to enter the data type, additional parameters are not required.

```
param order: list                // DataList of any type
param orders, deliveries: queue  // two DataQueues of any
type
param cost: table, VAT: real, complaints: stack // mixed declaration
DataTable and DataStack
```

Declare Optional Parameters

Declare optional parameters as follows:

```
param maxValue:real := 1.0 -> real // optional parameter
return z_uniform(0,maxValue)
```

Declare the Function Result

If a *Method* does not need any parameters, you just have to declare the data type of the result of the *Method*.

```
-> boolean // no parameters  
// or with parameters  
param v1,v2: integer, name: string -> boolean
```

See also

Declare Default Arguments

Declare the Function Result of a Method

Video on YouTube

<https://youtu.be/5t-wLNmKpbU?si=Q-8tGoRXDly3nd2i&t=34>

Declare Default Arguments

You can assign **default arguments**, also called **optional arguments**, or **optional parameters** in a *Method* in *SimTalk 2.0*.

Remarks

If the *Method* is called without the argument, the associated parameter will be set to the default value, which is defined in the *Method*.

```
param x: integer := 123
```

The *Method* can be called with or without argument. If the *Method* is called without argument, the parameter named x has the value 123 in the example above.

You can also assign **default arguments** to some of the parameters only. If a parameter has a default argument, all following parameters also have to have a **default argument**.

```
param a: string,  
      b, c: string := "",  
      d: boolean := true
```

The *Method* in the following example can be called with one to four arguments, but not without any argument at all, as no default argument was defined for the parameter named A.

Possible calls are:

```
Method("A")  
Method("A", "B")  
Method("A", "B", "C")  
Method("A", "B", "C", false)
```

Allowed Values for Default Arguments

You can specify the following values for default arguments:

- **Default arguments** only accept **constant values**, i.e., numbers, string constants, the boolean constants true and false, void, and pi.
- **Parameters** of data type *object*, *table*, *list*, *stack*, *queue*, and *any* can only have void as default argument.
- **Parameters** of data type *date* and *dateTime* cannot have a default argument at all.
- **Parameters** of data type *array* can have an empty *array* [] or an empty *JSON object* {} as default argument.

- **Reference parameters**, i.e., parameters declared by the keyword **byref**, cannot have a default argument.

See also

[Declare Parameters](#)

[Colors for Syntax Highlighting](#)

[Video on YouTube](#)

<https://youtu.be/5t-wLNmKpbU?si=4oreA6Pt8xKQtFuP&t=126>

Declare the Function Result of a Method

Mathematical functions return a **result**. A *Method* may also return a result.

Remarks

To make a *Method* work like a function, specify `->` (a minus sign and a greater-than sign) followed by the data type. If the function requires parameters, specify them in front of the declaration of the return value `->`.

Note

A function only returns **one** result.

Within the function you have to assign the result to the return value **result**. After the function has been executed, *Plant Simulation* returns the contents of this variable to the caller.

Compare these examples:

- A function without a parameter, calculating a *real* value, looks like this:

```
-> real
```

- A function with a parameter, returning a *string* value, looks like this:

```
param CustomerNo:integer
-> string
```

- A function with several parameters, returning a *table*, looks like this:

```
param orders,deliveries: list, delivery_date: time
-> table
```

Note

Instead of using *result*, you can also terminate the *Method* by using *return*, compare [Exiting Methods Using Return](#).

The data types *stack*, *queue*, *list*, and *table* do not behave like described above. At the beginning, the *local variable result* is empty. Before accessing *result* you either assign a value to *result* or you have to use the method **create**.